Prediksjon av vinkvalitet med maskinlæring

Sluttprosjekt i maskinlæring · Gruppe: Tor Seljestokken Nes, Sander Lund Kleppe · 29.10.2026

GitHub-repo: https://github.com/t184914/VinTesting · Nettside:https://vintestinggit.streamlit.app/

1: Beskriv problemet

Omfang

Mål. Vi har laget en nettside der brukeren stiller inn kjemiske målinger for en vin og får et anslag på om vinen vil bli vurdert som **lav**, **middels** eller **høy** kvalitet. Bak ligger en maskinlæringsmodell trent på kjemiske analyser av portugisiske «vinho verde»-viner. Prosjektet er et læringsprosjekt, og vi ønsket å gå gjennom hele livssyklusen til et ML-prosjekt: problemdefinisjon, data, modellering og utrulling.

Hvorfor maskinlæring. Det finnes ingen kjent formel som regner ut smakskvalitet fra kjemiske målinger. Sammenhengen mellom de 11 egenskapene og smaksvurderingen er sammensatt, og vi har mange merkede eksempler (smakspoeng fra smakspaneler). Det er en typisk situasjon for veiledet læring: modellen kan lære mønstre fra eksemplene i stedet for at vi må skrive regler for hånd.

Hvordan problemet løses i dag, og manuelt uten ML. Vinkvalitet vurderes i dag i hovedsak av smakspaneler og eksperter, mens kjemiske analyser brukes til kontroll. Uten maskinlæring må noen smake vinen, og det er tidkrevende, dyrt og subjektivt. Datasettet vi bruker er laget nettopp slik: smakere ga poeng til hver vin, og Cortez et al. (2009) undersøkte om kjemiske målinger kunne forutsi poengene.

Bruk og brukere. Løsningen er tenkt for små produsenter, hobbybryggere og nysgjerrige brukere som vil ha en første indikasjon på kvalitet ut fra laboratoriemålinger, før eller i stedet for en smaksvurdering. Den er ikke ment å erstatte smakere, og nettsiden sier at prediksjonen er et anslag.

Business objective. Vi har ikke laget et kommersielt produkt, så det er ingen lønnsomhetsmåling. Den tenkte gevinsten er å redusere antall smaksvurderinger som trengs, ved å bruke modellen til rask forhåndsvurdering.

Ressurser. Prosjektet er gjennomført av to studenter på omtrent en uke. Alt kjører på en vanlig bærbar PC uten GPU. Vi brukte gratis verktøy (Python, scikit-learn, Streamlit). Datasettet er lite, så beregningsbehovet er lavt.

Metrikker

ML-metrikker. Vi bruker **macro-F1** (gjennomsnittlig F1 over de tre klassene) som hovedmål. Klassene er ubalanserte (se avsnitt 2), og accuracy alene kan lure: en modell som alltid gjetter «middels» får 43,7 % riktig. Vi rapporterer også accuracy, precision og recall per klasse og confusion matrix.

Software-metrikker. Modellen er liten og gir prediksjon umiddelbart i nettsiden.

Suksesskriterier. Minimumskravet er at modellen slår dummy-baselinen (macro-F1 0,203) med god margin, og at den er minst like god som en enkel logistisk regresjon. Smakspoeng er subjektive, så vi

forventet ikke nær perfekt treffsikkerhet. Nettsiden skal i tillegg være forståelig og vise hvilke egenskaper som påvirker prediksjonen.

2: Data

Kilde og type data. Vi brukte datasettet *Wine Quality* fra UCI Machine Learning Repository (id 186), publisert av Cortez et al. (2009). Det er tabelldata og ble hentet med Python-pakken `ucimlrepo` og lagret som CSV i repoet (`data/`).

Mengde. Datasettet har 6497 viner (1599 røde og 4898 hvite). Hver vin har 11 kontinuerlige kjemiske egenskaper (fast syre, flyktig syre, sitronsyre, restsukker, klorider, fritt og totalt svoveldioksid, tetthet, pH, sulfater og alkohol). Det mangler ingen verdier. Datamengden er nok til modellene vi har brukt. Vi har ikke estimert hvor mye mer data som ville trengt for å forbedre resultatene vesentlig.

Labels. Labelen er kvalitetsscoren (0-10) gitt av smakere, og var ferdig i datasettet. Vi gjorde den om til tre klasser:

Klasse	Kvalitetspoeng
Lav	3-5
Middels	6
Høy	7-9

Vi valgte tre klasser fordi det er enklere å tolke for en bruker enn et tall som 5,8, og fordi det gir en tydelig confusion matrix. Konsistensen i labelene har vi ikke kunnet kontrollere selv: poengene er subjektive vurderinger, og grensene mellom nabopoeng er uskarpe. Å slå sammen poengene til klasser reduserer noe av denne støyen, men grensene (5/6 og 6/7) er vårt valg.

Personvern og etikk. Datasettet inneholder ingen personopplysninger, og heller ikke druetype, merke eller pris. Etisk er hovedpunktet at modellen ikke bør brukes som en fasit eller som erstatning for smakere, og at den bare gjelder portugisiske vinho verde-viner.

Rensing. Datasettet inneholdt 1177 duplikater (identiske rader). Vi fjernet dem, noe som ga **5320 unike viner**. Hvis duplikatene hadde blitt værende, kunne de samme radene havnet i både trenings- og testsettet, og resultatene ville blitt for gode.

Klassefordeling. 43,7 % middels, 37,4 % lav og 19,0 % høy.

Representasjon og skalering. Alle de 11 egenskapene brukes som tall. Vintype (rød/hvit) er lagt til som en egen feature (0 = rød, 1 = hvit). Vi gjorde ingen annen feature engineering. Egenskapene ble standardisert (StandardScaler) for logistisk regresjon. Trebaserte modeller trenger ikke skalering.

Deling. Dataene ble delt 80/20 med stratifisering (lik klassefordeling i begge deler) og fast `random_state=42`: 4256 rader til trening og 1064 til test. Testsettet ble ikke brukt under modellvalg eller tuning.

3: Modellering

Baseline. Vi har to baselines. Den første er en **dummy-klassifikator** som alltid gjetter vanligste klasse (accuracy 0,437, macro-F1 0,203): enhver reell modell må slå den. Den andre er en **logistisk regresjon** (med standardisering), som er enkel og forklarbar.

Modeller. I tillegg til baselines testet vi **Random Forest** og **Gradient Boosting**. Logistisk regresjon og Random Forest ble trent med `class_weight="balanced"` for å kompensere for de ubalanserte klassene.

Tuning. De tre modellene ble tunet med `GridSearchCV` (5-fold stratifisert kryssvalidering) på treningsdataene, med macro-F1 som mål.

Modell	Søkerom	Beste verdier
Logistisk regresjon	C: 0,01 / 0,1 / 1 / 10	C = 10
Random Forest	n_estimators: 200 / 400, max_depth: ingen / 10 / 20, min_samples_leaf: 1 / 3	400 trær, max_depth 20, min_samples_leaf 1
Gradient Boosting	n_estimators: 100 / 300, learning_rate: 0,05 / 0,1, max_depth: 2 / 3	300 trinn, learning_rate 0,1, max_depth 3

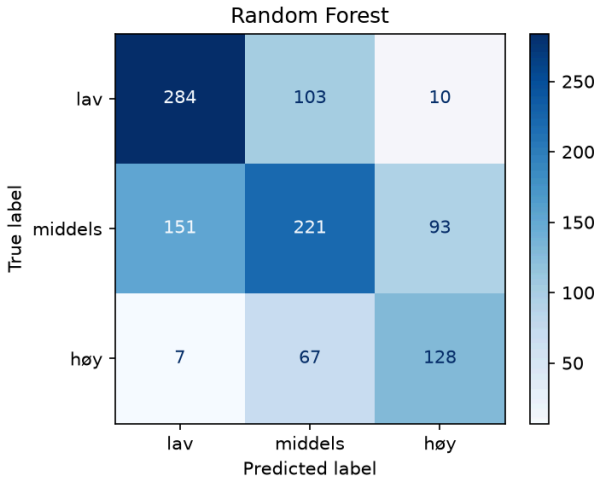
Resultater på testsettet:

Modell	CV macro-F1 (trening)	Test accuracy	Test macro-F1
Random Forest	0,619	0,595	**0,595**
Gradient Boosting	0,586	0,583	0,566
Logistisk regresjon	0,547	0,550	0,546
Dummy (vanligste klasse)	–	0,437	0,203

Random Forest ble valgt som endelig modell. Den har høyest macro-F1, og avstanden mellom kryssvalideringsscore (0,619) og testscore (0,595) er liten, noe som tyder på at modellen ikke er sterkt overtilpasset. Alle tre reelle modeller slår dummy-baselinen tydelig, så de kjemiske målingene inneholder reell informasjon om kvalitet. Den enkle logistiske regresjonen lå bare ca. 0,05 macro-F1 bak, noe som tyder på at en betydelig del av sammenhengen er tilnærmet lineær.

Resultatet for Random Forest per klasse:

Klasse	Precision	Recall	F1
Lav	0,64	0,72	0,68
Middels	0,57	0,48	0,52
Høy	0,55	0,63	0,59



Figur 1: Confusion matrix for Random Forest på testsettet (rader = faktisk klasse, kolonner = predikert klasse).

Confusion matrixen (Figur 1) viser at nesten alle feilene er forvekslinger mellom nabo-klasser. Av de 431 feilene på testsettet er 414 (ca. 96 %) mellom lav og middels eller mellom middels og høy: 151 middels-viner ble gjettet som lav, 103 lave som middels, 93 middels som høy og 67 høye som middels. Bare 17 viner (ca. 4 % av feilene) ble forvekslet mellom ytterpunktene: 10 lave ble gjettet som høye, og 7 høye ble gjettet som lave. Det tyder på at modellen fanger retningen på kvalitet, og at de fleste feilene kommer av at grensene mellom klassene er uskarpe.

Feilanalyse. Klassen «middels» er vanskeligst for alle modellene (recall 0,48 for Random Forest). Det er forventet: klassen ligger mellom de to andre, og grensene mellom kvalitet 5/6 og 6/7 er i praksis vilkårlige. Gradient Boosting hadde svakest gjenkjenning av «høy» (recall 0,43), mens logistisk regresjon fanget flest «høy»-viner (recall 0,72), men med lav precision (0,45), altså mange falske «høy». Dette illustrerer avveiningen mellom precision og recall for den sjeldne klassen. Se Figur 1 for hvilke klasser som forveksles.

Feature importance. Fra Random Forest:

Egenskap	Viktighet
Alkohol	0,155
Tetthet	0,108
Flyktig syre	0,099
Klorider	0,087
Totalt svoveldioksid	0,087
Sulfater	0,082
Fritt svoveldioksid	0,081

Egenskap	Viktighet
Sitronsyre	0,077
pH	0,076
Restsukker	0,076
Fast syre	0,069
Vintype (rød/hvit)	0,004

Alkohol er den viktigste egenskapen, etterfulgt av tetthet og flyktig syre. Vintype betyr nesten ingenting når de kjemiske målingene er kjent. Viktigheten er spredt ganske jevnt over de fleste egenskapene, så ingen enkeltegenskap bestemmer kvaliteten alene. Alkohol og tetthet er korrelerte, så importance bør tolkes forsiktig.

Begrensninger og mulige forbedringer.

- Kvalitetspoengene er subjektive, noe som setter en øvre grense for hvor godt kjemi alene kan forutsi kvalitet.
- Dataene gjelder bare portugisiske vinho verde-viner.
- Klassegrensene (3-5 / 6 / 7-9) er vårt valg, og andre grenser ville gitt andre resultater.
- Hyperparametersøket var lite. Beste `max_depth` for Random Forest (20) lå i ytterkant av søkerommet, så et bredere søk kan gi noe mer.

4: Deployment

Slik er modellen satt i drift. Den beste modellen (Random Forest) er lagret med `joblib` og brukes i en nettside laget med Streamlit. Brukeren velger vintype og justerer 11 skyveknapper (eller skriver inn tall i tallfelt), som er begrenset til min- og maksverdiene i treningsdataene og starter på gjennomsnittet. Nettsiden viser:

- predikert kvalitetsklasse og modellens sikkerhet, med sannsynlighetene for alle tre klasser
- feature importance for modellen
- en «hva hvis»-analyse som viser hvordan sannsynlighetene endrer seg når én egenskap varierer og resten holdes fast

«Hva hvis»-analysen endrer én egenskap om gangen, mens egenskapene i virkeligheten henger sammen. Ekstreme kombinasjoner kan derfor være urealistiske, og det står også på nettsiden.

Nettsiden er publisert på Streamlit Community Cloud: https://vintestinggit.streamlit.app/



Figur 2: Skjermbilde av nettsiden.

Hvordan prediksjonene brukes. Brukeren bruker prediksjonen som et anslag og en indikasjon, ikke som en endelig vurdering av vinen.

Monitorering og vedlikehold. Siden dette er et studentprosjekt har vi ikke satt opp automatisk monitorering. I en reell drift ville det vært viktig å logge inndata og prediksjoner for å oppdage om brukerne legger inn viner som ligner lite på treningsdataene (data drift), å sammenligne prediksjonene med faktiske smaksvurderinger når slike finnes, og å trene modellen på nytt hvis ytelsen faller. Versjonene av scikit-learn og de andre bibliotekene bør festes i `requirements.txt`, siden en lagret modell kan feile hvis den lastes med en annen versjon.

Planer for videre arbeid. Bredere hyperparametersøk, å prøve regresjon eller ordinal klassifisering, mer detaljert forklaring av enkeltprediksjoner (for eksempel SHAP), og opplasting av en CSV med flere viner.

Hvordan reprodusere resultatene.

1. Klon repoet og installer avhengighetene: `git clone https://github.com/t184914/VinTesting.git`, deretter `pip install -r requirements.txt`.
2. (Valgfritt) Hent dataene på nytt fra UCI: `python getData.py`. CSV-filene ligger allerede i `data/`.
3. (Valgfritt) Tren og evaluer modellene på nytt: `python TrainWine.py --red data/winequality-red.csv --white data/winequality-white.csv`. Dette lager `output/` med modell, resultattabell og figurer. Den ferdigtrente modellen ligger allerede i repoet.
4. Start nettsiden: `streamlit run App.py`.

5: Referanser

- Cortez, P., Cerdeira, A., Almeida, F., Matos, T. og Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*.
- UCI Machine Learning Repository: Wine Quality (id 186). https://archive.ics.uci.edu/dataset/186/wine+quality
- scikit-learn-dokumentasjonen (https://scikit-learn.org) og Streamlit-dokumentasjonen (https://docs.streamlit.io).

Bruk av AI-verktøy. Vi har brukt Claude (Anthropic) til å skrive utkast til treningsscriptet og Streamlit-appen, og til å skrive utkast til denne rapporten. Vi har også brukt det til å få forklart koden og begrepene underveis, for eksempel kryssvalidering og data leakage. Vi kjørte scriptene selv, hentet og la inn dataene, opprettet GitHub-repoet og deployet nettsiden på Streamlit Community Cloud. Resultatene i rapporten kommer fra våre egne kjøringer, og vi har selv satt inn lenken til nettsiden, skjermbildene og confusion matrixen.